



Storey — RLS Audit (full)

Date: 2026-05-19 · Supersedes RLS-AUDIT.md (which covered only migrations 001–008). Scope: every table — including tasks, task_updates, and the reworked material_transfers / attendance / daily_logs.

Audited: RLS enabled, S/I/U/D coverage, tenant isolation, role scoping, and — the focus this round — whether the **confirmation / approval workflows** are actually enforced by the database or only by the UI.

Severity summary

#	Finding	Tables	Severity
1	Confirmation workflow not DB-enforced — a submitter can self-confirm via the API	attendance, daily_logs, tasks	🔴 HIGH → ✅ FIXED (migration 20260519000000)
2	Transfer stage transitions unguarded — any allowed role can jump any stage	material_transfers	🟡 MEDIUM → ✅ FIXED (migration 20260519010000)
3	<code>tenants.plan</code> is user-mutable — a user can upgrade their own plan	tenants	🟡 MEDIUM → ✅ FIXED (migration 20260519020000)
4	Ledger alterable / direct-insert	material_transactions	🟢 LOW → ✅ append-only (migration 20260519030000); adjustment-approval workflow → TODO
5	No UPDATE/DELETE policy — mistakes are permanent	material_allocations	🟢 LOW

🔴 1. Confirmation workflows are UI-only, not DB-enforced

The "submit → Site Manager confirms" rule (built 2026-05-18 for attendance & daily logs, and the task submit→confirm) is enforced **only in the app UI**.

At the RLS level, the UPDATE policies are *role-broad*:

- `attendance_update` / `logs_update` / `tasks_update` all allow **supervisor**.
- RLS cannot restrict *which columns* a row update changes.

So a supervisor (or anyone with the anon key + their JWT) can, from the browser console or a script:

```
UPDATE attendance SET approval_status='confirmed', confirmed_by='<self>' WHERE ...;
UPDATE daily_logs SET approval_status='confirmed', confirmed_by='<self>' WHERE ...;
UPDATE tasks      SET status='done', confirmed_by='<self>' WHERE ...;
```

— self-confirming their own work, defeating the entire checks-and-balances layer. The Site Manager sign-off becomes optional in practice.

*Note: **receipts and site_expenses are NOT affected** — they're correctly designed with creator ≠ approver (store keeper / supervisor creates; only site_manager+ can update). That separation is the model to copy.*

Fix: a `BEFORE UPDATE` trigger on `attendance`, `daily_logs`, `tasks` that blocks a non-manager from moving a row into a confirmed state or setting `confirmed_by` — the same pattern as `enforce_profile_field_immutability`:

```
CREATE OR REPLACE FUNCTION public.enforce_confirmation_role()
RETURNS trigger LANGUAGE plpgsql SECURITY DEFINER SET search_path=public AS $$
BEGIN
    IF auth.role() = 'service_role' THEN RETURN NEW; END IF;
    IF public.my_role() IN ('superadmin','contractor','site_manager') THEN
        RETURN NEW;                -- managers may confirm
    END IF;
    -- everyone else: cannot set/raise confirmation fields
    IF TG_TABLE_NAME = 'tasks' THEN
        IF NEW.status = 'done' AND OLD.status <> 'done' THEN
            RAISE EXCEPTION 'Only a site manager can confirm a task';
        END IF;
        NEW.confirmed_by := OLD.confirmed_by;
        NEW.confirmed_at := OLD.confirmed_at;
    ELSE
        IF NEW.approval_status = 'confirmed' AND OLD.approval_status <> 'confirmed' THEN
            RAISE EXCEPTION 'Only a site manager can confirm this record';
        END IF;
        NEW.confirmed_by := OLD.confirmed_by;
        NEW.confirmed_at := OLD.confirmed_at;
    END IF;
    RETURN NEW;
END; $$
-- attach BEFORE UPDATE on attendance, daily_logs, tasks
```

2. Transfer stage transitions are unguarded

`mt_update` allows contractor / site_manager / store_keeper / supervisor to update any transfer on their sites. RLS does not enforce the **initiated → prepared → approved → received** order or who may do each stage — a supervisor could set `status='received'` directly, skipping approval.

In practice the UI gates this and the stock RPCs are the real money-path, but for integrity a `BEFORE UPDATE` trigger (or a status-transition check) should enforce legal transitions + role-per-stage. Medium priority — lower than #1 because transfers already route through the stock RPCs.

● 3. `tenants.plan` is user-mutable

`tenants_update` USING `owner_id = auth.uid()` lets a tenant owner update their tenant row. There is no column restriction, so an owner can run `UPDATE tenants SET plan='enterprise'` — a free self-upgrade. Harmless today (no billing logic), but it must be locked before any paid plan ships. Fix: a trigger freezing `plan` for non-superadmins, or move `plan` changes server-side.

● 4. Ledger direct-insert

`txn_insert` lets roles insert `material_transactions` rows directly, including a hand-picked `balance_after`. The app always uses the atomic, row-locked `record_material_transaction` RPC — but nothing stops a direct insert that pollutes the ledger. Low priority; consider restricting `txn_insert` to `service_role` only (the RPC is SECURITY DEFINER and bypasses RLS anyway).

● 5. `material_allocations` — no UPDATE/DELETE

Only SELECT + INSERT policies exist, so an allocation can never be corrected or removed once created. Likely unintentional. Add an UPDATE/DELETE policy for contractor/site_manager if allocations need correcting.

Verified good

- **RLS enabled on every table** (22 tables + `storage.objects` for site-photos).
 - **Tenant isolation** — every table filters `tenant_id = my_tenant_id()` (or a site-scoped equivalent); `storage.objects` scopes by the tenant path segment.
 - **INSERT policies pin `tenant_id`** via `WITH CHECK` — no cross-tenant inserts.
 - **`profiles`** — role/tenant_id locked by `enforce_profile_field_immutability` (the 2026-05-15 fix). Solid.
 - **`phone_verifications`** — the world-readable policy was removed (migration 009); only `service_role` writes, users read their own.
 - **`material_transactions`** — no UPDATE policy → immutable ledger. Correct.
 - **`receipts & expenses`** — creator ≠ approver separation done right.
 - **`pending_invites`** — full S/I/U/D, tenant-pinned, DELETE added in 009.
-

Recommended action order

1. **Finding 1 (HIGH) — confirmation guard:**  DONE — migration `20260519000000_confirmation_role_guard.sql` adds BEFORE UPDATE triggers on attendance, daily_logs and tasks. The confirmation layer is now enforced.
2. **Finding 2 — transfer stage transition guard:**  DONE — migration `20260519010000_transfer_stage_guard.sql` enforces the legal lifecycle and keeps approval manager-only.

3. ~~Finding 3~~ — lock `tenants.plan` :  DONE — migration `20260519020000_tenant_plan_lock.sql` freezes `plan` for non-superadmins.
4. ~~Finding 4~~ — ledger alterable:  DONE — migration `20260519030000_ledger_append_only.sql` removes the DELETE policy; the ledger is now strictly append-only (no UPDATE, no DELETE). The **adjustment-approval workflow** (Contractor signs off stock adjustments) is a feature — tracked on `docs/TODO.md` .
5. **Finding 5** — `material_allocations` has no UPDATE/DELETE. RESOLVED as a non-issue: allocations are posted records and *should* stay immutable, like the ledger. Over-allocation is corrected by posting a `return` transaction (unused material back to stock), not by editing history. The real gap is a missing **"Return material" action** in the app — logged to `docs/TODO.md` .

Each fix is a `BEFORE UPDATE` trigger — same proven pattern as the profiles immutability trigger. None require a table or policy rewrite.